Performance Engineering of a Spark Pipeline for NYC Yellow Taxi Analytics

ComPar 2026/27 – Project 1

Diogo Oliveira
André Nunes
77474
77077

ABSTRACT

We profiled and optimised a Kotlin Spark pipeline over the 146 M trips of the 2015 NYC Yellow Taxi data on a small Spark standalone cluster. The baseline lost its time in repeated scans, shuffles of whole trip rows and many small stages. Removing them one version at a time brought the five operations from 290 to 103 s (2.8×), and the operations that shuffled whole rows stopped spilling and their peak memory fell from 13.7 GB to 64 MB (O1).

KEYWORDS

Apache Spark, query optimisation, profiling, shuffle, scalability

1 INTRODUCTION

We use the 2015 NYC TLC Yellow Taxi trip records [2]: 12 monthly Parquet files (2.0 GB) with 146,039,231 trips and 19 columns, plus the zone lookup table (265 rows). We started on 2024 and moved to 2015 because it has 3.5 times more trips, enough for shuffles and spills to show up on our small cluster. We implemented the mandatory **M1** (ingestion and cleaning), **M2** (temporal demand per borough, day and hour) and **M3** (monthly zone statistics with windows), and the optional **O1** (fare anomalies per zone and hour) and **O4** (trip-type classification).

The workload is a large table reduced to a few thousand groups and joined with a tiny table, so each query should cost about one Parquet scan; extra scans, shuffles of full rows and spills come from how the query is written. We picked O1 and O4 because both compute statistics per bucket and then go back to every trip (O1 with a window, O4 with a join), so both shuffle whole rows, which the mandatory operations hardly do.

2 PIPELINE

We wrote the pipeline in Kotlin with the DataFrame API (agreed with the instructor instead of PySpark), which lets an operation mix Catalyst [1] queries with plain JVM code where that is faster (the driver-side windows of M3 v3, the columnar reader of M1 v3). Every operation implements the same `Analysis` interface and has numbered versions chosen on the command line, so each change is measured on its own against the baseline. M1 returns the cleaned DataFrame lazily and the other operations read from it; we do not cache it, since 142 M rows do not fit in 2 GB executors and each operation reads only its own columns. Every operation writes its result as Parquet (M1 writes its per-year cleaning statistics as CSV).

Figure 1 shows the physical plan of O1 (from Spark's event log) before and after its key change.

```
HashAggregate (+ Exchange,
   |   HashAggregate, write)
   +- BroadcastHashJoin
      :- Project
      :  +- Window
      :     +- Sort
      :        +- Exchange
      :           +- Filter
      :              +- Scan parquet
      +- BroadcastExchange
         +- Scan csv
```

```
HashAggregate (+ Exchange,
   |   HashAggregate, write)
   +- BroadcastHashJoin
      :- BroadcastHashJoin
      :  :- Filter
      :  :  +- Scan parquet
      :  +- BroadcastExchange
      :     +- HashAggregate
      :        +- Exchange
      :           +- HashAggregate
      :              +- Scan parquet
      +- BroadcastExchange
         +- Scan csv
```

Figure 1: Physical plan of O1 below the final aggregation and write, which are the same in both (Project nodes omitted): v0 with a Window over shuffled trips (left), v3 with a broadcast of the per-bucket statistics (right).

3 PROFILING AND BASELINE

Setup. The DI cluster was not available, so we measured on the course's Docker Spark standalone cluster (`docker-compose.yml`, image `spark:python3`: Spark 4.2.0, Scala 2.13, OpenJDK 21) in Docker Desktop on a laptop with an Intel Core Ultra 7 256V (4 performance + 4 efficiency cores), 16 GB RAM, NVMe SSD and Windows 11. One master and two workers with 2 cores and 2 GB each; the driver runs in the master container (client mode) and the jar, data and output are mounted from Windows. The only non-default setting is 2 GB of executor memory (`spark.executor.memory`, default 1 GB); AQE is on, as by default. We repeated every run on a second laptop running Linux with no virtual machine ("Linux" in Table 2 and Figure 4; machine and full results in Appendix A).

Methodology. `scripts/benchmark.sh` compiles the jar once and runs every version as a separate `spark-submit`, so no cache or JIT state carries over. Analytical operations run after M1 v3 and we count only their own jobs, with 20 s pauses between operations. Each version ran three times and we report the median; the three runs stay within about 10 % of it, so we treat smaller differences as noise. Our profiler reads the Spark REST API [3] (stages, tasks, CPU, GC, shuffle, spill, peak execution memory) into JSON, and `notebooks/benchmarks_report.ipynb` turns it into the tables and charts below. Every version produces the same output as the baseline unless stated.

What we found (Table 1, Figure 2):

- *Repeated scans.* M1 decodes every row twice (the per-year statistics and the second `count()`), and M3 three times, because `show()`, the count of flagged rows and the write are separate actions and nothing is cached. M3 is the slowest operation although its result has about 3,000 rows.
- *Shuffling whole rows.* The window of O1 shuffles every trip; with 2 GB executors the largest buckets spill 1.3 GB to disk and the

Table 1: Baseline (every operation at v0) on the cluster, median of three runs. "Heaviest stage" is the task time of the most expensive stage; the notebook lists the duration of every stage.

Op	wall (s)	stages	tasks	shuffle R / W (MB)	spill (MB)	heaviest stage (task time)
M1	40.9	15	135	0 / 0	0	scan, 68 s (2 such)
M2	21.6	11	71	1.4 / 0.7	0	scan, 65 s
M3	90.9	44	254	2.5 / 2.3	0	scan, 110 s (3 such)
O1	68.8	16	195	1,515 / 1,515	1,319	window, 143 s
O4	67.6	18	247	1,799 / 1,799	0	shuffle write, 136 s
Total	**289.8**					

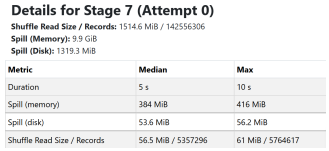


Figure 2: Spark UI (History Server) of the window stage of O1 v0, from an extra run with the event log on: all 142.6 M trips are shuffled (1.5 GB) and almost every task spills, by similar amounts.

window stage is the most expensive of the baseline. O4 shuffles 1.8 GB to join the bucket classes back to the trips.

- *Many small stages.* M3 works on a few thousand rows after the aggregation, but each window adds an Exchange and a Sort (44 stages).
- *No skew.* Each scan task reads one file and the shuffle keys have thousands of values. In the stages that do the work the slowest task takes at most 1.9× the median (the spilling window of O1), far from the 10× of real skew.

4 OPTIMISATION STUDY

Each subsection is one optimisation, in version order per operation. Figure 3 shows how time and memory change between versions and Table 2 summarises all of them.

4.1 M1 v1: one scan

(a) The baseline scans the data several times and casts every column of every row. **(b)** All counters can come from one pass, and most columns already have the right type. **(c)** One aggregation computes the total, the five criteria and the valid rows; the schema is checked once and only wrong types are cast. **(d)** 2.14× faster, with a third of the jobs and half the CPU time.

4.2 M1 v2: bitmask per row

(a) Seven conditional sums each evaluate their own condition on every row. **(b)** One integer per row with a bit per criterion should be cheaper. **(c)** v2 counts per (year, bitmask), a few dozen groups, and the driver derives the counters. **(d)** 4 % less CPU, same time.

4.3 M1 v3: direct columnar reader

(a) Spark still turns every column batch into rows before counting. **(b)** Reading the column vectors ourselves should leave only the cost of decoding Parquet. **(c)** Each task reads the batches of one file, builds the bitmasks in plain loops and counts in an array; the driver adds the 12 arrays. **(d)** 2 stages, 13 tasks, no shuffle, a quarter less CPU and 1.44× faster than v2 (3.2× over v0). About 30 % of the task time is still spent waiting for the files, read through the mount from Windows.

4.4 M2 v1: projection and broadcast

(a) The baseline reads every column and joins the zone table. **(b)** Selecting five columns and forcing a broadcast should read less. **(c)** v1 does both. **(d)** Same plan, stages, shuffle and time as v0: Catalyst was already pruning and broadcasting the 12 KB table.

4.5 M2 v2: no global sort

(a) The `orderBy` before the partitioned write adds a range-partitioning shuffle, but **(b)** `partitionBy(borough)` only needs rows grouped by borough. **(c)** v2 uses `repartition(borough)`. **(d)** Two stages fewer and half the shuffle, but 6 % slower (22.9 vs 21.6 s).

4.6 M2 v3: aggregate before the join

(a) The zone join is applied to every trip. **(b)** Aggregating first by (zone, day, hour) shrinks its input to a few thousand rows. **(c)** v3 aggregates per zone, then joins and aggregates per borough. **(d)** A third of the peak memory, about the same time.

4.7 M2 v4: no join

(a) v3 still has a join and a second aggregation. **(b)** Zone→borough is a 265-entry map, not a join. **(c)** v4 builds the map on the driver, packs (borough, day, hour) into 11 bits and aggregates once. **(d)** Peak memory 1.4 GB→32 MB, same time.

4.8 M2 v5–v7: less work per row

(a) v4 evaluates a `CASE WHEN` chain and `unix_timestamp` on every row. **(b)** Less work per row and fewer stages should help. **(c)** v5 uses an array lookup and divides once per group, v6 writes with `coalesce(1)`, v7 reads the CSV on the driver. **(d)** All stay within the range of the runs (21.1–21.9 s): the removed stages were cheap. M2 is bound by its scan and no version moves its time, so we keep v4, which needs the least memory.

4.9 M3 v1: cache the result

(a) `show()`, the count of flagged rows and the write each read and aggregate the trips again. **(b)** If the small final table is cached, the later actions cost almost nothing. **(c)** v1 caches the result (about 3,000 rows) and broadcasts the zone table. **(d)** One heavy scan instead of three: 1.54× faster, under half the CPU. The cache keeps 200 shuffle partitions, so tasks go from 254 to about 1,000 and the reported peak memory to 6.4 GB for tiny data.

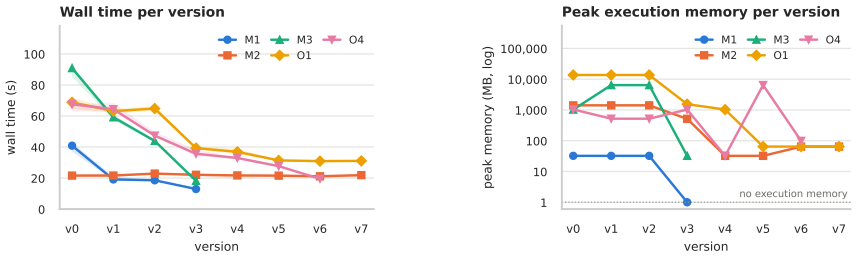


Figure 3: Wall time (left) and peak execution memory (right, log scale) of every version, on the cluster with the 2015 data. Memory is the sum of the task peaks per stage, as Spark reports it; M1 v3 bypasses Spark's operators and reports none.

4.10 M3 v2: integer keys

(a) `date_format` builds a string for each of 146 M rows and the grouping key has strings. **(b)** Grouping by integers and formatting after the aggregation moves that work to 3,000 rows. **(c)** v2 groups by (`PULocationID`, year · 100 + month). **(d)** Another 1.35×, and CPU time halves again.

4.11 M3 v3: windows on the driver

(a) Each window still needs its own Exchange and Sort for 3,000 rows. **(b)** A loop on the driver should be cheaper than scheduling dozens of stages. **(c)** v3 collects the aggregated rows and computes the rank, the 3-month rolling share and the deviation flag (Welford [4]) in Kotlin. **(d)** 48→8 stages, about 1,000→38 tasks and 5.0× faster than v0, the largest gain of the project. We also found that the original flag could never fire: with the current month inside the 3-month window, no month can be more than 1.155 standard deviations from the mean, below the 1.5 threshold. All versions now use the three *previous* months.

4.12 O1 v1 and v2: projection and broadcast

(a) The baseline carries all columns through several projections and joins the zone table. **(b)** One projection and a forced broadcast should move less data. **(c)** v1 uses one `select`, v2 adds `broadcast(zones)`. **(d)** Same plan, shuffle and spill: Catalyst was already doing both, and the 6–9 % gain is about the spread of the runs of v0.

4.13 O1 v3: aggregation instead of a window

(a) A window cannot combine on the map side, so every trip is shuffled and large buckets spill. **(b)** The mean and standard deviation per bucket are an aggregation with at most 263×24 groups. **(c)** v3 uses `groupBy(zone, hour)`, which aggregates before the shuffle, and broadcasts the 6,312 thresholds back to the trips. **(d)** Shuffle 1.5 GB→3 MB, no spill, nine times less peak memory and 1.65× faster than v2 (1.75× over v0). The price, a second scan, costs far less than shuffling and spilling.

4.14 O1 v4 and v5: integer keys and an array instead of the join

(a) The broadcast join hashes and probes every trip, and the aggregation carries strings. **(b)** Integer keys and an array lookup should be cheaper. **(c)** v4 groups by one integer key per bucket and per rate (borough resolved on the driver); v5 puts the thresholds in a 67 KB array read with `element_at`, so no join is left. **(d)** 6 % and 15 % faster than the previous version; after v5 peak memory is 64 MB.

4.15 O1 v6 and v7: less arithmetic per trip

(a) v5 updates a floating-point average and converts every distance to km, per trip and in both scans. **(b)** The anomaly rate can come from two integer counts, and comparing in fare per mile gives the same anomalies without converting. **(c)** v6 uses integer counters; v7 compares per mile. **(d)** 30.9 and 31.1 s against 31.4 s for v5, within noise: O1 now costs about its two scans, 2.23× faster than v0.

4.16 O4 v1: one projection

(a) Both branches share a wide DataFrame built with chained `withColumn`. **(b)** One `select` of seven columns should make the shuffled rows smaller. **(c)** v1 projects them and derives hour and month in one step. **(d)** 10 % less shuffle and half the peak memory, same time: the 1.6 GB shuffle of every trip is still there.

4.17 O4 v2: broadcast the bucket classes

(a) The join of trips and classified buckets is a sort-merge join that shuffles all 142 M trips. **(b)** The planner cannot know the aggregation has at most 6,312 rows, but we do. **(c)** v2 wraps the classes in `broadcast()`. **(d)** Shuffle 1.6 GB→1.7 MB and a quarter less time.

4.18 O4 v3: integer keys

(a) The join and aggregations use a two-column key and strings (`date_format` for every trip). **(b)** Integers move the string work to the 48 output rows. **(c)** v3 replaces the two-column key and the strings with one integer key. **(d)** 30 % less CPU, 1.33× over v2.

4.19 O4 v4: an array instead of the join

(a) The broadcast join still hashes every trip and three floating-point averages are updated per row. **(b)** An array indexed by the key and integer counters should be cheaper. **(c)** v4 uses integer sums and an 8 KB array of zone types read with `element_at`. **(d)** 8 % faster; peak memory 1 GB→32 MB, as no hash table is built.

4.20 O4 v5: one scan

(a) Every version so far reads the data twice: to classify the buckets and to compute the monthly statistics. **(b)** Trips with the same (month, zone, hour) share a class, and counts, sums and sums of

Table 2: Summary of all optimisations (2015 data, median and range of three runs; mem: peak execution memory; CPU times quoted in the text are in the notebook; Linux: median on the second machine). The best versions (M1 v3, M2 v4, M3 v3, O1 v6, O4 v6) take 103.2 s instead of 289.8 s for v0 (2.81×), and 135.3 s instead of 390.7 s on Linux (2.89×).

Op	version	wall (s)	range (s)	vs v0	mem (MB)	Linux (s)
M1	v0 baseline	40.9	37.1–42.9	1.00	32	49.3
	v1 one scan	19.1	18.6–21.0	2.14	32	22.9
	v2 bitmask	18.6	18.4–18.6	2.20	32	20.8
	v3 columnar reader	12.9	12.7–13.8	3.17	0	16.3
M2	v0 baseline	21.6	20.9–22.3	1.00	1,412	30.3
	v1 projection, broadcast	21.6	20.9–22.1	1.00	1,412	29.3
	v2 no global sort	22.9	22.5–22.9	0.94	1,412	28.7
	v3 aggregate first	22.1	21.5–22.1	0.98	515	29.4
	v4 no join	21.7	20.4–21.8	1.00	32	27.9
	v5 array lookup	21.6	21.3–22.4	1.00	32	28.6
	v6 coalesce(1)	21.1	20.3–21.5	1.02	64	29.9
	v7 CSV on driver	21.9	20.7–22.3	0.99	64	28.6
M3	v0 baseline	90.9	85.5–92.4	1.00	1,028	133.2
	v1 cache result	59.2	57.4–60.8	1.54	6,412	65.8
	v2 integer keys	43.9	43.0–44.9	2.07	6,412	40.3
	v3 windows on driver	18.2	17.4–18.3	4.99	32	23.4
O1	v0 baseline	68.8	64.6–71.8	1.00	13,703	89.9
	v1 projection	63.1	61.8–64.6	1.09	13,703	86.1
	v2 + broadcast	64.8	63.0–66.5	1.06	13,703	86.1
	v3 aggregation, no window	39.3	38.4–40.2	1.75	1,544	58.4
	v4 integer keys	36.9	35.4–39.0	1.86	1,029	48.5
	v5 array, no join	31.4	31.2–31.4	2.19	64	42.8
	v6 integer counters	30.9	30.4–31.8	2.23	64	41.6
	v7 per-mile compare	31.1	30.9–31.7	2.21	64	41.9
O4	v0 baseline	67.6	62.9–67.8	1.00	1,040	88.0
	v1 projection	64.4	63.0–68.4	1.05	520	87.0
	v2 broadcast classes	47.4	47.2–49.8	1.43	520	68.9
	v3 integer keys	35.7	34.1–38.4	1.89	1,029	45.2
	v4 array, no join	33.0	32.0–33.2	2.05	32	42.2
	v5 one scan	27.7	27.1–28.6	2.44	6,450	38.3
	v6 finish on driver	19.5	19.4–19.6	3.47	99	26.1
All	best versions	103.2		2.81		135.3

squares add up, so mean and standard deviation can come from about 38,000 partial groups. **(c)** v5 aggregates the trips once into them, caches them and finishes with two small Spark jobs. **(d)** A third less CPU and 16 % less time, but 22 stages, 1,500 tasks and 6.4 GB of reported peak memory, as in M3 v1.

4.21 O4 v6: finish on the driver

(a) v5 spends stages and tasks to reduce 38,000 rows. **(b)** As in M3 v3, plain code on the driver should be cheaper. **(c)** v6 packs (month, zone, hour) into one integer, aggregates the trips once, collects the 38,000 rows (2 MB) and finishes in Kotlin. **(d)** 6 stages, 38 tasks, 99 MB and 3.47× faster than v0; results match v0 up to the last digits of the standard deviation.

4.22 Summary

In Table 2, the largest gains come from removing work: extra scans, shuffles of whole rows and stages for a few thousand rows (M1 v1, M3 v3, O1 v3, O4 v2, O4 v6). Memory falls when a join or hash table goes away (M2 v4, O1 v5, O4 v4), while caching a small result in 200 partitions shows up as a memory peak even when it saves time (M3 v1, O4 v5).

5 ANALYSIS

We ran the optimised pipeline (the best versions, but M1 v2, chosen before the repeated runs) with `scaling_2015.sh`, three runs per condition (Figure 4).

Dataset scale. The time grows much less than the data. With 10 % of the trips the pipeline takes 75.2 s, 78 % of the 96.6 s of the full data, and with 20 % it takes 83 % (Linux: 71 and 75 % of 111.2 s). A straight line through the three points gives about 73 s that do not depend on the fraction and 24 s that do. Two things make the fixed part so large: scheduling the jobs and stages, reading the zone table and the work on the driver cost the same for any input, and `--sample` draws the rows *after* reading, so every Parquet file is still opened, read through the mount and decoded; only the work after the scan shrinks. That is why O4, which still aggregates every trip, drops by 31 %, while M1, which is almost only the read, drops by 18 %.

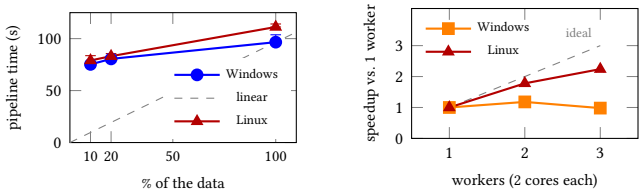


Figure 4: Optimised pipeline, median of 3 runs (2 for Windows with 3 workers; bars: range). Left: time vs. data fraction (2 workers). Right: speedup (full data).

Cluster size. On Windows, 2 workers make the pipeline 1.18× faster than one (efficiency 59 %) and 3 workers no faster (0.98×, 33 %); on Linux the same runs reach 1.78× and 2.24× (89 and 75 %). Three causes explain the deviation from the ideal line. The fixed cost on the driver and in scheduling does not shrink with more executors: fitting $t = a + b/w$ to the Linux times gives $a \approx 34$ s, 17 % of the time with one worker. The scans have one task per file (13 tasks), so with 4 or 6 cores the last wave is almost empty. Above all on Windows, the workers are containers in one virtual machine with 8 cores (half of them efficiency cores) and read every file through the same mount: the executor CPU time for the same work grows from 157 to 224 and 309 s with 1, 2 and 3 workers, against 12 % more on Linux, where the 16 cores are not shared.

6 DISCUSSION

What is left. Each operation now costs roughly the scans it needs: one pass for M1, M2, M3 and O4, two for O1. Every operation reads the same files again; materialising the cleaned data once (a large enough cache or a narrower Parquet file) would avoid this.

Limitations. Three runs per version separate real changes from noise only above about 10 %, so for M2, O1 v1/v2 and O1 v6/v7 we rely on the plan metrics. Peak memory sums the task peaks per stage, too high for stages with many tasks (M3 v1, O4 v5), and Spark does not report Parquet input bytes on the cluster, so we infer scans from the plans. The baseline total sums separate runs per operation. **With more time** we would run more repetitions on the DI cluster.

ACKNOWLEDGMENTS

We used two AI tools during the project, Claude Code (Anthropic) and Antigravity (Google), mostly to help with research, with the code and with writing this report. We ran and checked all the measurements ourselves.

INDIVIDUAL CONTRIBUTIONS

Member	Work	%
Diogo Oliveira	M1, M3, benchmark, report	45
André Nunes	M2, O1, O4, profiler, Docker benchmark	55

COMMENTS AND SUGGESTIONS

Access to the DI cluster during the project would have let us measure scaling on separate machines instead of containers sharing one laptop.

REFERENCES

[1] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 1383–1394.

[2] NYC Taxi & Limousine Commission. 2024. TLC Trip Record Data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page. Accessed November 2026.

[3] The Apache Software Foundation. 2026. Apache Spark 4.2.0 Documentation: SQL Performance Tuning and Monitoring and Instrumentation. https://spark.apache.org/docs/4.2.0/. Accessed November 2026.

[4] B. P. Welford. 1962. Note on a Method for Calculating Corrected Sums of Squares and Products. *Technometrics* 4, 3 (1962), 419–420.

A MEASUREMENTS ON THE LINUX MACHINE

The same benchmark (`benchmark.sh --all --runs 3` and `scaling_2015.sh`) on the second laptop of Section 3: Intel Core Ultra 7 155H (6 performance + 8 efficiency + 2 low-power cores), 32 GB RAM, NVMe SSD, Fedora Linux 45 with Docker Engine and no virtual machine, Spark 4.2.1, and the same cluster (two workers with 2 cores and 2 GB, client mode). Stages, tasks, shuffle, spill and memory match the Windows machine, because they depend on the data and the plan; only the times differ.

Table 3: Linux machine: baseline (every operation at v0), median of three runs; same columns as Table 1.

Op	wall (s)	stages	tasks	shuffle R / W (MB)	spill (MB)	heaviest stage (task time)
M1	49.3	15	135	0 / 0	0	scan, 80 s (2 such)
M2	30.3	11	71	1.4 / 0.7	0	scan, 88 s
M3	133.2	44	254	2.5 / 2.3	0	scan, 162 s (3 such)
O1	89.9	16	195	1,515 / 1,515	1,319	window, 202 s
O4	88.0	18	247	1,799 / 1,799	0	shuffle write, 173 s
Total	**390.7**					

Table 4: Linux machine: optimised pipeline (Section 5), median and range of three runs; CPU is the executor CPU time.

workers	data	time (s)	range (s)	speedup	efficiency	CPU (s)
1	100 %	197.7	188.4–199.1	1.00	100 %	309
2	100 %	111.2	111.2–114.1	1.78	89 %	323
3	100 %	88.3	87.4–90.5	2.24	75 %	345
2	20 %	83.1	82.5–85.6			222
2	10 %	79.0	77.8–83.6			203

Table 5: Linux machine: all optimisations, median and range of three runs; CPU is the executor CPU time and mem the peak execution memory. The best versions take 135.3 s instead of 390.7 s for v0 (2.89×).

Op	version	wall (s)	range (s)	vs v0	CPU (s)	mem (MB)
M1	v0 baseline	49.3	46.9–49.8	1.00	127.2	32
	v1 one scan	22.9	22.6–23.1	2.15	52.2	32
	v2 bitmask	20.8	20.4–21.4	2.37	45.4	32
	v3 columnar reader	16.3	15.5–16.4	3.02	37.5	0
M2	v0 baseline	30.3	29.1–31.9	1.00	75.0	1,412
	v1 projection, broadcast	29.3	29.2–31.0	1.03	74.2	1,412
	v2 no global sort	28.7	28.5–30.3	1.06	75.4	1,412
	v3 aggregate first	29.4	28.9–29.4	1.03	77.3	515
	v4 no join	27.9	27.6–28.0	1.09	73.3	32
	v5 array lookup	28.6	28.4–31.8	1.06	75.1	32
	v6 coalesce(1)	29.9	29.8–30.5	1.01	74.2	64
	v7 CSV on driver	28.6	28.3–29.9	1.06	76.1	64
M3	v0 baseline	133.2	127.9–135.9	1.00	354.2	1,028
	v1 cache result	65.8	63.1–67.6	2.02	131.3	6,412
	v2 integer keys	40.3	40.3–40.8	3.31	66.1	6,412
	v3 windows on driver	23.4	23.2–23.8	5.69	57.7	32
O1	v0 baseline	89.9	88.4–91.2	1.00	270.9	13,703
	v1 projection	86.1	85.2–87.5	1.04	265.8	13,703
	v2 + broadcast	86.1	85.5–86.6	1.04	266.6	13,703
	v3 aggregation, no window	58.4	54.6–60.9	1.54	164.5	1,544
	v4 integer keys	48.5	46.5–49.5	1.85	134.1	1,029
	v5 array, no join	42.8	40.4–44.0	2.10	113.8	64
	v6 integer counters	41.6	39.5–43.1	2.16	113.8	64
	v7 per-mile compare	41.9	40.6–44.1	2.15	111.3	64
O4	v0 baseline	88.0	87.7–89.6	1.00	281.5	1,040
	v1 projection	87.0	86.7–87.7	1.01	275.7	520
	v2 broadcast classes	68.9	65.7–69.3	1.28	188.7	520
	v3 integer keys	45.2	44.9–48.2	1.95	127.4	1,029
	v4 array, no join	42.2	41.8–45.8	2.09	116.2	32
	v5 one scan	38.3	38.2–39.7	2.30	83.3	6,450
	v6 finish on driver	26.1	26.0–26.5	3.37	68.8	99
All	best versions	135.3		2.89		